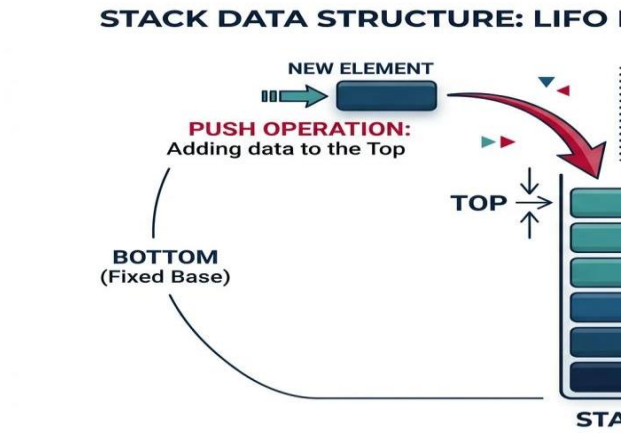
Stacks

A **Stack** is a linear data structure in which insertion and deletion of elements take place only at **one end**, called the **TOP**.

A stack follows the **LIFO (Last In, First Out)** principle.



1. Definition of Stack

A **stack** is a linear data structure where:

- Elements are inserted from the **TOP**.
- Elements are deleted from the **TOP**.
- The element inserted **last** is removed **first**.
- Therefore, Stack follows **LIFO – Last In, First Out**.

Real-life example

Think of a stack of plates:

```
 ┌─────┐
TOP → │Plate│ ← Last inserted
```

```
├─────┤
│Plate│
├─────┤
│Plate│
└─────┘
```

The last plate placed on the stack is the first plate removed.

2. Basic Terminology

TOP

TOP indicates the position of the element that is currently at the top of the stack.

For an array implementation, initially:

TOP = -1

This indicates that the stack is empty.

Stack Overflow

Overflow occurs when we try to insert an element into a stack that is already full.

if TOP == MAX - 1

  Overflow

Stack Underflow

Underflow occurs when we try to delete an element from an empty stack.

if TOP == -1

  Underflow

3. Stack Operations

The major stack operations are:

Operation	Meaning
Push	Insert an element
Pop	Delete/remove the top element
Peek/Top	View the top element without removing it
isEmpty	Check whether stack is empty
isFull	Check whether stack is full

4. Implementation of Stack Using Array

Suppose:

MAX = 5

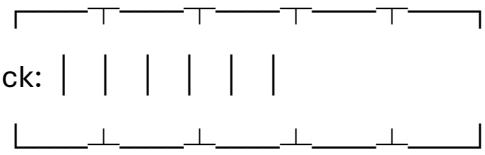
We create:

stack[5]

TOP = -1

Initially:

Index: 0 1 2 3 4



TOP = -1

---

5. PUSH Operation

Push means inserting an element into the stack.

Algorithm: PUSH

PUSH(STACK, ITEM)

1. If TOP == MAX - 1

    Print "Stack Overflow"

    Exit

2. TOP = TOP + 1

3. STACK[TOP] = ITEM

4. Stop

Example

Push:10 20 30

Initially:

TOP = -1

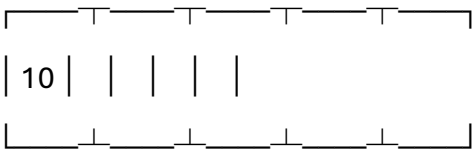
Push 10

TOP = TOP + 1

TOP = 0

STACK[0] = 10

Index: 0 1 2 3 4



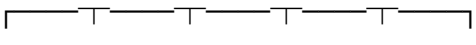
↑

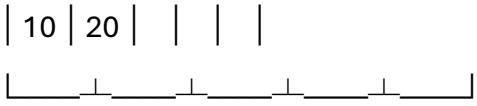
TOP

Push 20

TOP = 1

STACK[1] = 20

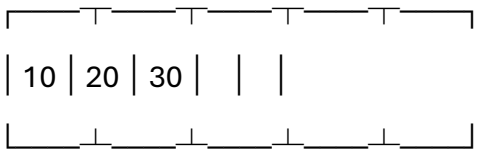




↑

TOP

Push 30



↑

TOP

6. POP Operation

Pop means removing the element from the top of the stack.

Algorithm: POP

POP(STACK)

1. If TOP == -1

    Print "Stack Underflow"

    Exit

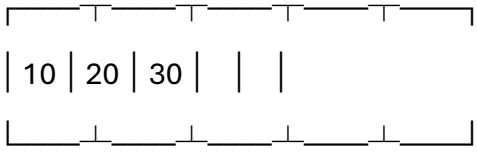
2. ITEM = STACK[TOP]

3. TOP = TOP - 1

4. Return ITEM

Example

Current stack:



↑

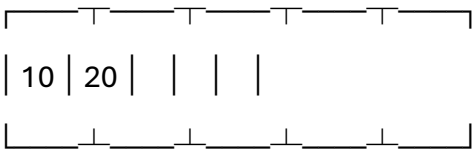
TOP

Perform:

POP()

The value 30 is removed.

Now:



↑

TOP

Returned value:

30

7. PEEK Operation

Peek returns the top element without removing it.

Algorithm

PEEK(STACK)

1. If TOP == -1

    Print "Stack is Empty"

    Exit

2. Return STACK[TOP]

For:

10 20 30

↑

TOP

PEEK() returns:

30

But 30 remains in the stack.

8. Stack Implementation Using Python

Without using Python's built-in append() and pop():

```
MAX = 5
stack = [None] * MAX
top = -1
def push(item):
  global top
  if top == MAX - 1:
    print("Stack Overflow")
    return
  top = top + 1
  stack[top] = item
def pop():
  global top
  if top == -1:
    print("Stack Underflow")
    return None
  item = stack[top]
  top = top - 1
  return item
def peek():
  if top == -1:
    print("Stack is Empty")
    return None
  return stack[top]
```

Example:

```
push(10)
push(20)
push(30)
print(pop())
print(peek())
```

Output:

30

20

9. Applications of Stack

Stacks are widely used in computer science.

1. Expression Evaluation

Stacks are used to evaluate:

- Infix expressions
- Prefix expressions
- Postfix expressions

2. Expression Conversion

Stacks are used to convert:

Infix → Postfix

Infix → Prefix

3. Function Calls

Programming languages use a **call stack** to manage function calls.

4. Recursion

Recursive function calls are maintained using a stack.

5. Undo/Redo

Applications such as text editors use stacks for undo and redo operations.

6. Browser History

Back/forward operations can be implemented using stacks.

7. Parentheses Matching

Stacks can check whether brackets are balanced:

{ [( )] }

10. Expression Notations

There are three common ways of writing arithmetic expressions:

1. **Infix**
2. **Prefix**
3. **Postfix**

Consider:

A + B

Infix

Operator is written **between operands**.

A + B

Prefix

Operator is written **before operands**.

+ A B

Postfix

Operator is written **after operands**.

A B +

11. Infix Expression

Infix is the normal mathematical notation used by humans.

Examples:

A + B

A - B

A * B

(A + B) * C

A + B * C

The operator comes between two operands.

A + B

↑ ↑

operand

12. Prefix Expression

In prefix notation, the operator comes **before** its operands.

It is also called **Polish notation**.

Example:

A + B

Prefix:

+ A B

Another example:

(A + B) * C

Prefix:

* + A B C

13. Postfix Expression

In postfix notation, the operator comes **after** its operands.

It is also called **Reverse Polish Notation (RPN)**.

Example:

A + B

Postfix:

A B +

Example:

(A + B) * C

Postfix:

A B + C *

14. Why Use Prefix and Postfix?

Infix expressions require rules such as:

- Operator precedence
- Associativity
- Parentheses

For example:

A + B * C

We know multiplication has higher precedence, so:

A + (B * C)

Prefix and postfix expressions remove much of this ambiguity.

For example:

Infix: A + B * C

Postfix: A B C * +

Prefix: + A * B C

15. Operator Precedence

When evaluating expressions, operators are processed according to precedence.

Operator	Meaning	Precedence
()	Parentheses	Highest
^	Power	3

Operator	Meaning	Precedence
* / %	Multiplication, Division, Modulus	2
+ -	Addition, Subtraction	1

Higher number means higher precedence.

For example:

A + B * C

* has higher precedence than +.

Therefore:

A + (B * C)

16. Associativity

When operators have the same precedence, **associativity** determines the evaluation order.

Left-to-right

For:

A - B + C

Evaluate:

(A - B) + C

Operators +, -, *, /, % generally have **left-to-right associativity**.

Right-to-left

Exponentiation ^ is generally treated as **right-to-left associative** in expression-conversion algorithms.

Example:

A ^ B ^ C

is interpreted as:

A ^ (B ^ C)

17. Evaluation of Postfix Expression

Postfix expressions are very easy to evaluate using a stack.

Rules

Scan the expression from **left to right**.

If the symbol is an operand:

Push it onto the stack.

If the symbol is an operator:

1. Pop the **top operand** → operand2
2. Pop the **next operand** → operand1
3. Perform:

operand1 operator operand2

4. Push the result back onto the stack.

At the end, the stack contains the final answer.

18. Postfix Evaluation Example

Evaluate:

2 3 4 * +

Step 1: Read 2

Operand → Push.

Stack:

2

Step 2: Read 3

Push.

Stack:

2

3

Step 3: Read 4

Push.

Stack:

2

3

4

Step 4: Read *

Pop 4:

operand2 = 4

Pop 3:

operand1 = 3

Calculate:

3 * 4 = 12

Push 12.

Stack:

2

12

Step 5: Read +

Pop 12:

operand2 = 12

Pop 2:

operand1 = 2

Calculate:

2 + 12 = 14

Push 14.

Stack:

14

Answer

14

19. Algorithm for Postfix Evaluation

EvaluatePostfix(expression)

1. Create an empty stack.

2. Scan expression from left to right.

3. If the symbol is an operand:

    PUSH(symbol)

4. Else if the symbol is an operator:

    operand2 = POP()

    operand1 = POP()

    result = operand1 operator operand2

    PUSH(result)

5. After scanning the complete expression:

    answer = POP()

6. Return answer.

Complexity

If there are n symbols:

Time Complexity = O(n)

Space Complexity = O(n)

20. Evaluation of Prefix Expression

Prefix is evaluated using a stack, but the scanning direction is **right to left**.

Rules

Scan the prefix expression from **right to left**.

If operand:

PUSH(operand)

If operator:

1. Pop first operand → operand1
2. Pop second operand → operand2
3. Calculate:

operand1 operator operand2

4. Push result.

21. Prefix Evaluation Example

Evaluate:

+ 2 * 3 4

This represents:

2 + (3 * 4)

Scan from **right to left**:

4

3

*

2

+

Read 4

Stack:

4

Read 3

Stack:

4

3

Read *

Pop:

operand1 = 3

operand2 = 4

Calculate:

3 * 4 = 12

Push:

Stack:

12

Read 2

Stack:

12

2

Read +

Calculate:

2 + 12 = 14

Final:

14

---

22. Infix Expression Evaluation Using Stack

An **infix expression** is one in which the operator is written between its operands.

Example:
A + B * C
(10 + 5) * 2

To evaluate an infix expression, we use **two stacks**:

1. **Operand stack** – stores numbers/operands.
2. **Operator stack** – stores operators such as +, -, *, /, %, ^.

Algorithm

Input: An infix expression E
Output: Evaluated value of E

1. Initialize an empty **operand stack** and **operator stack**.
2. Scan the expression from **left to right**.
3. If the scanned element is an **operand**, push it onto the operand stack.
4. If it is **(**, push it onto the operator stack.
5. If it is **)**, repeatedly:
    - Pop an operator from the operator stack.
    - Pop the required operands.
    - Perform the operation.
    - Push the result back onto the operand stack.
    - Continue until (is encountered, then remove (.
6. If the scanned element is an **operator**:
    - While the operator stack is not empty and the top operator has **higher or equal precedence**, perform that operation.
    - Push the current operator onto the operator stack.
7. After scanning the complete expression, perform all remaining operations in the operator stack.
8. The final value in the operand stack is the **result**.

Operator Precedence

^	Highest
*, /, %	Medium
+, -	Lowest

^ is generally **right-associative**, while +, -, *, /, % are **left-associative**.

Example: Evaluate (10 + 5) * 2 - 8 / 4

Scanned Symbol	Operand Stack	Operator Stack	Action
(	—	(	Push (

Scanned Symbol	Operand Stack	Operator Stack	Action
10	10	(	Push operand
+	10	(+	Push operator
5	10, 5	(+	Push operand
)	15	—	Evaluate 10 + 5
*	15	*	Push operator
2	15, 2	*	Push operand
-	30	-	Evaluate 15 * 2
8	30, 8	-	Push operand
/	30, 8	- /	Push /
4	30, 8, 4	- /	Push operand
End	28	—	Evaluate / and -

Calculation

$$(10 + 5) \times 2 - \frac{8}{4}$$
$$= 15 \times 2 - 2$$
$$= 30 - 2 = \boxed{28}$$

Final Answer: 28.

Python implementation of infix expression evaluation using stacks, along with complexity and another worked example.

1. Python Implementation

```
def precedence(op):
    if op == '^':
        return 3
    elif op in ('*', '/', '%'):
        return 2
    elif op in ('+', '-'):
        return 1
    return 0
```

```
def apply_operator(operators, operands):
    op = operators.pop()
    right = operands.pop()
    left = operands.pop()
    if op == '+':
        result = left + right
    elif op == '-':
        result = left - right
    elif op == '*':
        result = left * right
    elif op == '/':
```

```
        result = left / right
    elif op == '%':
        result = left % right
    elif op == '^':
        result = left ** right
    operands.append(result)
def evaluate_infix(expression):
    operands = []
    operators = []
    i = 0
    while i < len(expression):
        ch = expression[i]
        # Ignore spaces
        if ch == ' ':
            i += 1
            continue
        # If operand is a number
        if ch.isdigit():
            num = 0
            while i < len(expression) and
expression[i].isdigit():
                num = num * 10 +
int(expression[i])
                i += 1
            operands.append(num)
            continue
        # Opening parenthesis
        elif ch == '(':
            operators.append(ch)
        # Closing parenthesis
        elif ch == ')':
            while operators and operators[-1] !=
'(':
                apply_operator(operators,
operands)
            operators.pop()   # Remove '('
        # Operator
        elif ch in '+-*/%^':
            while (operators and operators[-1] !=
'(' and
                   precedence(operators[-1]) >=
precedence(ch)):
                apply_operator(operators,
operands)
            operators.append(ch)
        i += 1


    # Apply remaining operators
    while operators:
        apply_operator(operators, operands)


    return operands[-1]
# Example
expression = "(10 + 5) * 2 - 8 / 4"
```

print("Result:", evaluate_infix(expression))

Output:

Result: 28.0

2. Complexity Analysis

Let **n** be the number of symbols in the expression.

Time Complexity: O(n)

- Each operand is pushed onto the operand stack once.
- Each operator is pushed and popped from the operator stack at most once.
- Each symbol is scanned once.

Therefore:

$$\boxed{T(n) = O(n)}$$

Space Complexity: O(n)

In the worst case, the stacks may contain n elements.

$$\boxed{S(n) = O(n)}$$

Complexity	Value
Time Complexity	**O(n)**
Space Complexity	**O(n)**
Best Case	O(n)
Average Case	O(n)
Worst Case	O(n)

3. Another Example

Consider:

$$8 + 3 \times (10 - 6)/2$$

Step-by-step evaluation

Symbol	Operand Stack	Operator Stack	Action
8	8	—	Push operand
+	8	+	Push operator
3	8, 3	+	Push operand
*	8, 3	+, *	Push operator
(	8, 3	+, *, (	Push (
10	8, 3, 10	+, *, (	Push operand
-	8, 3, 10	+, *, (, -	Push operator
6	8, 3, 10, 6	+, *, (, -	Push operand
)	8, 3, 4	+, *	Evaluate 10 - 6
/	8, 3, 4	+, *, /	Push /
2	8, 3, 4, 2	+, *, /	Push operand

Symbol	Operand Stack	Operator Stack	Action
End	8, 3, 2	+	Evaluate /, *, +

Calculation

$$8 + 3 \times (10 - 6)/2$$
$$= 8 + 3 \times 4/2$$
$$= 8 + 12/2$$
$$= 8 + 6$$
$$\boxed{14}$$

So, the Python code gives:

expression = "8 + 3 * (10 - 6) / 2"

print(evaluate_infix(expression))

Output:

14.0

Key point: The stack-based approach automatically follows **parentheses and operator precedence**, so multiplication/division are performed before addition/subtraction where appropriate.

24. Important Difference: Prefix vs Postfix Evaluation

Feature	Prefix	Postfix
Scanning direction	Right → Left	Left → Right
Data structure	Stack	Stack
Operand	Push	Push
Operator	Pop 2 operands	Pop 2 operands
Result	Push back	Push back
Example	+ 2 3	2 3 +

Inter-conversion of Infix, Prefix and Postfix Expressions

Expression conversion is an important application of **Stack Data Structure**. The three common forms are:

Notation	Example	Operator Position
Infix	A + B	Between operands
Prefix	+ A B	Before operands
Postfix	A B +	After operands

For example:

$$A + B * C$$

can be represented as:

- **Infix:** A + B * C
- **Prefix:** + A * B C
- **Postfix:** A B C * +

1. Infix → Postfix

Algorithm

Input: Infix expression E
Output: Equivalent postfix expression

1. Create an empty **operator stack**.
2. Scan the infix expression from **left to right**.
3. If the symbol is an **operand**, add it to the postfix expression.
4. If the symbol is **(**, push it onto the stack.
5. If the symbol is **)**:
    - Pop operators from the stack and add them to the postfix expression until (is encountered.
    - Remove (from the stack.
6. If the symbol is an **operator**:
    - While the stack is not empty and the top operator has higher or equal precedence, pop it and add it to the postfix expression.
    - Push the current operator onto the stack.
7. After scanning all symbols, pop all remaining operators from the stack.
8. The resulting expression is the **postfix expression**.

Example

Convert:

A + B * C

Symbol	Stack	Postfix
A	—	A
+	+	A
B	+	AB
*	+ *	AB
C	+ *	ABC
End	—	ABC*+

Result

Infix : A + B * C

Postfix : A B C * +

Because * has higher precedence than +.

---

2. Infix → Prefix

A convenient stack-based method is:

Algorithm

1. Reverse the infix expression.
2. Replace every (with) and every) with (.
3. Convert the resulting expression into **postfix**.
4. Reverse the obtained postfix expression.
5. The result is the **prefix expression**.

Example

Convert:

A + B * C

Step 1: Reverse

C * B + A

Step 2: Convert to postfix

C B * A +

Step 3: Reverse

+ A * B C

Result

Infix : A + B * C

Prefix : + A * B C

---

3. Postfix → Infix

This conversion uses an **operand stack**.

Algorithm

1. Create an empty stack.
2. Scan the postfix expression from **left to right**.
3. If the symbol is an operand, push it onto the stack.
4. If the symbol is an operator:
    - Pop the top two operands.
    - Let the first popped operand be operand2.
    - Let the second popped operand be operand1.
    - Form:

(operand1 operator operand2)

    - Push the resulting expression onto the stack.
5. After scanning the complete expression, the stack contains the required infix expression.

Example

Convert:

A B C * +

Symbol	Stack	Operation
A	A	Push
B	A, B	Push
C	A, B, C	Push
*	A, (B*C)	B*C
+	(A+(B*C))	A+(B*C)

Result

Postfix : A B C * +

Infix : A + (B * C)

---

4. Postfix → Prefix

Algorithm

1. Create an empty stack.
2. Scan the postfix expression from **left to right**.
3. If the symbol is an operand, push it.
4. If the symbol is an operator:
    - Pop the top two elements.
    - Let them be operand2 and operand1.
    - Form:

operator operand1 operand2

5. Push the resulting expression back onto the stack.
6. The final stack element is the prefix expression.

Example

Convert:

A B C * +

Symbol	Stack	Operation
A	A	Push
B	A, B	Push
C	A, B, C	Push
*	A, B, * B C	Create *BC
+	+ A * B C	Create +A*BC

Result

Postfix : A B C * +

Prefix : + A * B C

---

5. Prefix → Infix

Algorithm

1. Create an empty stack.
2. Scan the prefix expression from **right to left**.
3. If the symbol is an operand, push it onto the stack.
4. If the symbol is an operator:
    - Pop the top two operands.
    - Form:

(operand1 operator operand2)

5. Push the resulting expression onto the stack.

6. The final element is the infix expression.

Example

Convert:

+ A * B C

Scan from right to left:

Symbol	Stack	Operation
C	C	Push
B	C, B	Push
*	(B*C)	Create
A	(B*C), A	Push
+	(A+(B*C))	Create

Result

Prefix : + A * B C

Infix : A + (B * C)

---

6. Prefix → Postfix

Algorithm

1. Create an empty stack.
2. Scan the prefix expression from **right to left**.
3. If the symbol is an operand, push it.
4. If the symbol is an operator:
    - Pop the top two operands.
    - Form:

operand1 operand2 operator

5. Push the resulting expression.
6. The final stack element is the postfix expression.

Example

Convert:

+ A * B C

Scanning right to left:

C → B → * → A → +

For *:

B C *

For +:

A B C * +

Result

Prefix : + A * B C

Postfix : A B C * +

---

7. Postfix → Prefix and Prefix → Postfix

These conversions can be summarized as:

Postfix → Prefix

Postfix: A B C * +

Prefix : + A * B C

Prefix → Postfix

Prefix : + A * B C

Postfix: A B C * +

Both use a **stack**.

---

8. Complete Conversion Summary

For the expression:

A + B * C

Conversion	Result
Infix	A + B * C
Prefix	+ A * B C
Postfix	A B C * +

Thus:

$$\boxed{A + B * C \iff +A * BC \iff ABC * +}$$

---

9. Complexity Analysis

Let **n** be the number of operands/operators in the expression.

For all standard stack-based conversions:

Time Complexity

Each symbol is scanned a constant number of times.

$$\boxed{O(n)}$$

Space Complexity

The stack may contain up to n elements in the worst case.

$$\boxed{O(n)}$$

Conversion	Time	Space
Infix → Postfix	**O(n)**	**O(n)**
Infix → Prefix	**O(n)**	**O(n)**
Postfix → Infix	**O(n)**	**O(n)**
Postfix → Prefix	**O(n)**	**O(n)**
Prefix → Infix	**O(n)**	**O(n)**
Prefix → Postfix	**O(n)**	**O(n)**

Important Exam Point

Stack is essential because operators and operands must be temporarily stored while maintaining **operator precedence, associativity, and parentheses**.

Example: Convert Infix to Postfix

Convert the following infix expression:

A + B * (C - D) / E

We scan the expression **from left to right** and use a stack for operators.

Symbol	**Action**	**Stack**	**Postfix**
A	Operand → add to output	—	A
+	Push operator	+	A
B	Operand → add to output	+	AB
*	Push (* has higher precedence)	+ *	AB
(	Push	+ * (	AB
C	Operand → add	+ * (	ABC
-	Push	+ * (-	ABC
D	Operand → add	+ * (-	ABCD
)	Pop until (	+ *	ABCD-
/	Pop * because same precedence, then push /	+ /	ABCD-*
E	Operand → add	+ /	ABCD-*E
End	Pop remaining operators	—	ABCD-*E/+

Final Answer

Infix:

A + B * (C - D) / E

Postfix:

A B C D - * E / +

or

ABCD-*E/+

Key Point

For an operator:

While stack top has higher or equal precedence:

Pop operator → add to postfix

Then:

Push current operator

So, when / is encountered, * is popped first because * and / have the **same precedence** and are **left-associative**.

one more example

Example 2: Infix to Postfix

Convert the following infix expression:

A * (B + C) - D / (E - F)

We scan from **left to right** using a stack.

Symbol	**Action**	**Stack**	**Postfix**
A	Operand → add to output	—	A
*	Push operator	*	A
(	Push	* (	A
B	Operand → add	* (	AB
+	Push operator	* (+	AB
C	Operand → add	* (+	ABC
)	Pop +, remove (	*	ABC+
-	Pop *, then push -	-	ABC+*
D	Operand → add	-	ABC+*D
/	Push /	- /	ABC+*D
(	Push	- / (	ABC+*D

E	Operand → add	- / (	ABC+*DE
-	Push	- / (-	ABC+*DE
F	Operand → add	- / (-	ABC+*DEF
)	Pop -, remove (	- /	ABC+*DEF-
End	Pop /, then -	—	ABC+*DEF/-

Final Answer

Infix:

A * (B + C) - D / (E - F)

Postfix:

A B C + * D E F - / -

or

ABC+*DEF/-

Verification

The expression can be viewed as:

(A * (B + C)) - (D / (E - F))

Postfix for each part:

A B C + * → ABC+*

D E F - / → DEF-/

Finally, apply -:

ABC+* DEF-/ -

Therefore:

ABC+*DEF/-

Merge Sort?

Merge Sort is a **divide-and-conquer sorting algorithm** used to arrange elements in ascending or descending order.

The basic idea is:

Divide → Sort → Merge

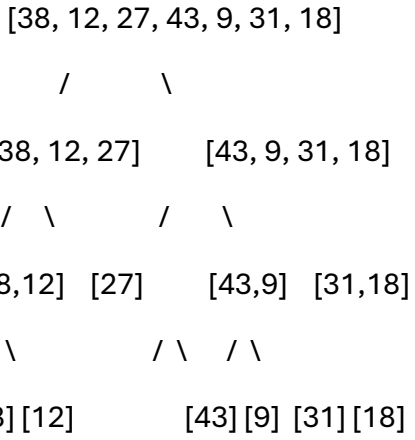
It divides the array into smaller subarrays until each subarray contains only one element. A single element is already sorted. Then, the sorted subarrays are repeatedly **merged** to produce larger sorted subarrays until the complete array becomes sorted.

Example

Consider:

A = [38, 12, 27, 43, 9, 31, 18]

Division phase



Now every subarray has one element.

Merging phase

[38] + [12] → [12, 38]

[43] + [9] → [9, 43]

[31] + [18] → [18, 31]

[12, 38] + [27] → [12, 27, 38]

[9, 43] + [18,31] → [9, 18, 31, 43]

[12,27,38] + [9,18,31,43]

→ [9,12,18,27,31,38,43]

---

2. Why is Merge Sort called Divide and Conquer?

Merge Sort follows three major steps:

1. Divide

Divide the array into two approximately equal halves.

mid = (low + high) // 2

2. Conquer

Recursively apply Merge Sort to both halves.

MERGE_SORT(left half)

MERGE_SORT(right half)

3. Combine

Merge the two sorted halves into one sorted array.

MERGE(left, right)

Thus:

Divide

↓

Solve left part

↓

Solve right part

↓

Merge

3. Main Components of Merge Sort

Merge Sort essentially consists of **two algorithms/functions**:

1. **MERGE_SORT()** — divides the array recursively.
2. **MERGE()** — combines two sorted subarrays.

For example:

MERGE_SORT(A)

|

↓

Divide A

/ \

/ \

Left Right

| |

↓ ↓

Sort Sort

\ /

\ /

↓ ↓

MERGE

↓

Sorted Array

4. Merge Operation

The **merge operation** is the most important part of Merge Sort.

Suppose we have two already sorted arrays:

Left = [12, 27, 38]

Right = [9, 18, 31, 43]

We compare the first elements:

12 vs 9

9 is smaller, so put 9 into the result.

Result = [9]

Next:

12 vs 18

12 is smaller.

Result = [9, 12]

Next:

27 vs 18

Put 18.

Result = [9, 12, 18]

Continue:

27 vs 31 → 27

38 vs 31 → 31

38 vs 43 → 38

Finally, 43 remains.

Result = [9, 12, 18, 27, 31, 38, 43]

Important point

At every step, **only the smallest front element of the two sorted subarrays is selected**.

5. Merge Sort Algorithm — Pseudocode

Algorithm 1: MERGE_SORT

MERGE_SORT(A, low, high)

1. If low < high then
2. mid = (low + high) // 2
3. MERGE_SORT(A, low, mid)
4. MERGE_SORT(A, mid + 1, high)
5. MERGE(A, low, mid, high)
6. Stop

Algorithm 2: MERGE

MERGE(A, low, mid, high)

1. Create two temporary arrays:

    Left = A[low ... mid]

    Right = A[mid+1 ... high]

2. Set:

    i = 0

    j = 0

    k = low

3. While i < length(Left) AND j < length(Right):

    If Left[i] <= Right[j] then

        A[k] = Left[i]

        i = i + 1

    Else

        A[k] = Right[j]

        j = j + 1

    k = k + 1

4. While i < length(Left):

    A[k] = Left[i]

    i = i + 1

    k = k + 1

5. While j < length(Right):

    A[k] = Right[j]

    j = j + 1

    k = k + 1

6. Stop

6. Detailed Description of the Algorithm

Let's understand each step.

Step 1: Check whether division is required

if low < high

If the subarray contains more than one element, it needs to be divided.

If:

low == high

there is only one element, so it is already sorted.

This is called the **base condition** of recursion.

Step 2: Find the middle

mid = (low + high) // 2

The array is divided into:

Left = A[low ... mid]

Right = A[mid+1 ... high]

Step 3: Sort the left half

MERGE_SORT(A, low, mid)

The left half is again divided into two parts.

This process continues recursively.

Step 4: Sort the right half

MERGE_SORT(A, mid + 1, high)

The right half is also recursively divided and sorted.

Step 5: Merge

After both halves have been sorted:

MERGE(A, low, mid, high)

The two sorted halves are combined.

This process continues upward through the recursion tree until the entire array is sorted.

7. Complete Example

Consider:

A = [38, 12, 27, 43, 9, 31, 18]

Division

[38, 12, 27, 43, 9, 31, 18]

↓

[38, 12, 27] [43, 9, 31, 18]

↓ ↓

[38,12] [27] [43,9] [31,18]

↓ ↓

[38] [12] [43] [9] [31] [18]

Now merging starts.

First level

[38] + [12] → [12,38]

[43] + [9] → [9,43]

[31] + [18] → [18,31]

Second level

[12,38] + [27]

→ [12,27,38]

and

[9,43] + [18,31]

→ [9,18,31,43]

Final merge

[12,27,38] + [9,18,31,43]

Comparisons:

12 vs 9 → 9

12 vs 18 → 12

27 vs 18 → 18

27 vs 31 → 27

38 vs 31 → 31

38 vs 43 → 38

Remaining → 43

Therefore:

[9, 12, 18, 27, 31, 38, 43]

8. Python Implementation

```
def merge_sort(arr):
    # Base condition
    if len(arr) <= 1:
        return arr
    # Divide
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]
    # Recursively sort both halves
    left = merge_sort(left)
    right = merge_sort(right)
    # Merge
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Main program
arr = [38, 12, 27, 43, 9, 31, 18]
print("Original Array:", arr)
sorted_arr = merge_sort(arr)
print("Sorted Array:", sorted_arr)
```

Output

Original Array: [38, 12, 27, 43, 9, 31, 18]

Sorted Array: [9, 12, 18, 27, 31, 38, 43]

9. Complexity Analysis

The major advantage of Merge Sort is that its running time is **O(n log n)**.

Time Complexity

Case	Complexity	Reason
Best Case	**O(n log n)**	Array is still divided and merged
Average Case	**O(n log n)**	Same divide-and-merge structure
Worst Case	**O(n log n)**	Same number of levels and merging work

Why O(n log n)?

There are two components.

Number of levels

At every level, the array is divided approximately into half.

n

n/2

n/4

n/8

...

1

The number of divisions is:

$\log_2 n$

At every level, merging requires approximately:

O(n)

Therefore:

Total = O(n) × O(log n)

= O(n log n)

10. Recurrence Relation

Merge Sort can also be represented mathematically as:

T(n) = 2T(n/2) + O(n)

where:

- 2T(n/2) → sorting the two halves
- O(n) → merging the two halves

Using the Master Theorem:

T(n) = O(n log n)

---

11. Space Complexity

Merge Sort generally requires additional memory for temporary arrays.

Space Complexity = O(n)

The recursive function also requires a recursion stack of approximately:

O(log n)

But the additional arrays used during merging dominate the space requirement.

Therefore:

Auxiliary Space = O(n) for the usual array implementation.

12. Important Properties of Merge Sort

Property	**Merge Sort**
Technique	Divide and Conquer
Best Time	O(n log n)
Average Time	O(n log n)
Worst Time	O(n log n)
Space	O(n)
Stable	**Yes**
In-place	Generally **No**
Recursive	Yes
Adaptive	Generally No
Suitable for linked lists	**Yes**
Suitable for external sorting	**Yes**

Stable sorting

Merge Sort can preserve the relative order of equal elements.

For example:

(A, 50), (B, 30), (C, 50)

After sorting:

(B, 30), (A, 50), (C, 50)

A remains before C, so the algorithm is stable.

13. Comparison with Other Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable	Basic Approach
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Repeated swapping
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Usually No	Select minimum
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Insert into sorted part
Merge Sort	**O(n log n)**	**O(n log n)**	**O(n log n)**	O(n)	**Yes**	Divide and conquer
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)*	Usually No	Partition

Merge Sort vs Quick Sort

This is one of the most important comparisons.

Feature	Merge Sort	Quick Sort
Approach	Divide & Conquer	Divide & Conquer
Best	O(n log n)	O(n log n)
Average	O(n log n)	O(n log n)
Worst	**O(n log n)**	$O(n^2)$
Extra space	O(n)	Usually O(log n)
Stable	**Yes**	Usually No
In-place	Usually No	Usually Yes
Practical speed	Good	Often very fast
Worst-case guarantee	**Yes**	No, without special strategies

18. Advantages of Merge Sort

1. Guaranteed O(n log n)

Even in the worst case:

O(n log n)

2. Stable

Equal elements retain their relative ordering.

3. Suitable for large datasets

Its predictable complexity makes it useful for large amounts of data.

4. Good for linked lists

Merge operations can be performed efficiently on linked lists without requiring random access.

5. Useful for external sorting

When data is too large to fit into main memory, Merge Sort is widely useful because sorted chunks can be processed and merged.

6. Parallelization

Different halves can be sorted independently, making Merge Sort suitable for parallel processing.

19. Disadvantages of Merge Sort

1. Extra memory

The standard implementation requires:

O(n)

additional space.

2. Not usually in-place

Unlike Heap Sort, the standard array implementation requires temporary storage.

3. More overhead for small arrays

For small datasets, simpler algorithms such as Insertion Sort can sometimes be faster because of lower overhead.

4. Recursive overhead

The traditional implementation uses recursion.

Quick Sort?

Quick Sort is a **divide-and-conquer sorting algorithm**. It selects one element as a **pivot** and rearranges the array so that:

- Elements **smaller than the pivot** come before it.
- Elements **greater than the pivot** come after it.
- The pivot reaches its correct position.

Then Quick Sort is recursively applied to the left and right parts.

Basic idea

Choose Pivot → Partition → Recursively Sort Left and Right

For example:

Array = [38, 12, 27, 43, 9, 31, 18]

Suppose we choose 18 as the pivot.

After partitioning, one possible arrangement is:

[12, 9] 18 [38, 27, 43, 31]

↓ ↓ ↓

smaller pivot greater

Then we recursively sort:

[12, 9] [38, 27, 43, 31]

↓ ↓

[9, 12] [27, 31, 38, 43]

Final result:

[9, 12, 18, 27, 31, 38, 43]

2. Why is Quick Sort called Divide and Conquer?

Quick Sort follows three steps:

1. Divide

Choose a pivot and partition the array around it.

Smaller elements | Pivot | Larger elements

2. Conquer

Recursively sort the left and right partitions.

3. Combine

Unlike Merge Sort, **Quick Sort does not require a separate merge step**.

Once the left and right parts are sorted, the entire array is sorted.

This is an important difference:

Merge Sort divides first and merges later, whereas Quick Sort partitions around a pivot and sorts the resulting parts.

3. Important Terms

Pivot

The element selected for partitioning.

For example:

[25, 10, 30, 15, 20]

↑

Pivot

The pivot could be:

- First element
- Last element
- Middle element
- Random element
- Median/median-of-three

The choice of pivot strongly affects Quick Sort's performance.

Partition

The process of rearranging elements around the pivot.

After partitioning:

Elements < Pivot | Pivot | Elements > Pivot

The pivot is now in its **final sorted position**.

4. Example of Partitioning

Consider:

A = [10, 7, 8, 9, 1, 5]

Let's choose the **last element 5 as pivot**.

[10, 7, 8, 9, 1, 5]

↑

pivot

We examine the elements one by one.

Compare 10 with 5

10 > 5

No change.

Compare 7 with 5

7 > 5

No change.

Compare 8 with 5

8 > 5

No change.

Compare 9 with 5

9 > 5

No change.

Compare 1 with 5

1 < 5

Move it to the left side.

After partitioning, we get:

[1, 7, 8, 9, 10, 5]

Then the pivot is placed between the smaller and larger elements:

[1] 5 [7, 8, 9, 10]

So:

Pivot position = 1

Now Quick Sort is recursively applied to:

[1]

and

[7, 8, 9, 10]

5. Quick Sort Algorithm

We will use the **Lomuto partition scheme**, where the last element is selected as the pivot.

Algorithm: QUICK_SORT

QUICK_SORT(A, low, high)

1. If low < high:
2. p = PARTITION(A, low, high)
3. QUICK_SORT(A, low, p - 1)
4. QUICK_SORT(A, p + 1, high)

5. Stop

6. Partition Algorithm

PARTITION(A, low, high)

1. pivot = A[high]

2. i = low - 1

3. For j = low to high - 1:

4. If A[j] <= pivot:

5. i = i + 1

6. Swap A[i] and A[j]

7. Swap A[i + 1] and A[high]

8. Return i + 1

7. Detailed Explanation of Partition Algorithm

Let's understand the variables.

pivot = A[high]

The last element is selected as the pivot.

For example:

A = [10, 7, 8, 9, 1, 5]

pivot = 5

i

i = low - 1

i keeps track of the position where the next element smaller than or equal to the pivot should be placed.

j

j scans the array from:

low → high - 1

Whenever:

A[j] <= pivot

we increment i and swap:

A[i] ↔ A[j]

At the end, we put the pivot after all smaller elements.

8. Complete Partition Example

Consider:

A = [10, 7, 8, 9, 1, 5]

Pivot:

5

Initially:

i = -1

Step 1

j = 0

10 > 5

No swap.

i = -1

Step 2

j = 1

7 > 5

No swap.

Step 3

j = 2

8 > 5

No swap.

Step 4

j = 3

9 > 5

No swap.

Step 5

j = 4

1 <= 5

Increment i:

i = 0

Swap:

A[0] ↔ A[4]

Array becomes:

[1, 7, 8, 9, 10, 5]

Finally, swap:

A[i+1] ↔ A[high]

That means:

A[1] ↔ A[5]

Result:

[1, 5, 8, 9, 10, 7]

Here, the pivot 5 is at its correct position.

Note: The exact arrangement of elements greater than the pivot is not important; what matters is that elements on the left are ≤ pivot and elements on the right are > pivot.

9. Complete Quick Sort Working

Consider:

[10, 7, 8, 9, 1, 5]

First partition

Pivot = 5

[1] 5 [8, 9, 10, 7]

Now sort the right side:

[8, 9, 10, 7]

Choose 7 as pivot.

[] 7 [9, 10, 8]

Then sort:

[9, 10, 8]

Choose 8:

[] 8 [10, 9]

Then:

[] 9 [10]

Final:

[1, 5, 7, 8, 9, 10]

10. Python Implementation — Lomuto Partition

```
def partition(arr, low, high):
    # Select last element as pivot
    pivot = arr[high]
    i = low - 1
    # Traverse the array
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            # Swap
            arr[i], arr[j] = arr[j], arr[i]
    # Place pivot at its correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
def quick_sort(arr, low, high):
    if low < high:
        # Partition the array
        p = partition(arr, low, high)
```

```
        # Sort left part
        quick_sort(arr, low, p - 1)
        # Sort right part
        quick_sort(arr, p + 1, high)
# Main program
arr = [38, 12, 27, 43, 9, 31, 18]
print("Original Array:", arr)
quick_sort(arr, 0, len(arr) - 1)
print("Sorted Array:", arr)
```

Output

Original Array: [38, 12, 27, 43, 9, 31, 18]

Sorted Array: [9, 12, 18, 27, 31, 38, 43]

11. Another Python Implementation — Easier to Understand

For teaching beginners, the following implementation is conceptually simpler:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    left = []
    middle = []
    right = []
    for x in arr:
        if x < pivot:
            left.append(x)
        elif x == pivot:
            middle.append(x)
        else:
            right.append(x)
    return quick_sort(left) + middle + quick_sort(right)
arr = [38, 12, 27, 43, 9, 31, 18]
print("Original Array:", arr)
print("Sorted Array:", quick_sort(arr))
```

Output:

Original Array: [38, 12, 27, 43, 9, 31, 18]

Sorted Array: [9, 12, 18, 27, 31, 38, 43]

Important

This version is **easy to understand**, but it uses additional arrays and therefore is not the usual in-place Quick Sort implementation.

For an **algorithm/data structures exam or implementation**, the first implementation using partition() is generally more appropriate.

12. Complexity Analysis

Quick Sort has an important difference from Merge Sort:

Its performance depends on how well the pivot divides the array.

Best Case

The pivot divides the array into approximately equal halves.

n

/ \

n/2 n/2

/ \ / \

n/4 ...

The recurrence is:

T(n) = 2T(n/2) + O(n)

Therefore:

Best Case = O(n log n)

13. Average Case

On average, Quick Sort produces reasonably balanced partitions.

Therefore:

Average Case = O(n log n)

This is one reason Quick Sort is widely used in practice.

14. Worst Case

The worst case occurs when the pivot repeatedly becomes the **smallest or largest element**.

For example, consider an already sorted array:

[1, 2, 3, 4, 5, 6, 7]

If we always select the last element as pivot:

Pivot = 7

Partition becomes:

[1,2,3,4,5,6] 7 []

Then:

Pivot = 6

Again:

[1,2,3,4,5] 6 []

This produces a highly unbalanced recursion tree.

The recurrence becomes:

T(n) = T(n-1) + O(n)

Therefore:

Worst Case = $O(n^2)$

15. Summary of Time Complexity

Case	**Time Complexity**
Best	**O(n log n)**
Average	**O(n log n)**
Worst	**$O(n^2)$**

This is the most important complexity point about Quick Sort.

16. Space Complexity

The partitioning can be performed **in-place**, so Quick Sort does not normally require an additional array of size n.

However, recursion requires stack space.

Average recursion depth

O(log n)

Worst recursion depth

O(n)

Therefore:

Case	Auxiliary Stack Space
Average	O(log n)
Worst	O(n)

With careful implementation and pivot strategies, worst-case stack behavior can be reduced.

17. Pivot Selection

Pivot selection is extremely important.

Common approaches are:

1. First element

pivot = A[low]

Simple but can perform poorly on sorted data.

2. Last element

pivot = A[high]

Easy to implement, as in our example.

3. Middle element

pivot = A[(low + high) // 2]

Can avoid some poor patterns.

4. Random pivot

Choose a random element.

This reduces the likelihood of repeatedly encountering highly unbalanced partitions for many input patterns.

5. Median-of-three

Choose the median of:

first element

middle element

last element

This can provide better pivot choices for some practical datasets.

18. Quick Sort vs Merge Sort

This is a very important comparison.

Feature	Quick Sort	Merge Sort
Technique	Divide & Conquer	Divide & Conquer
Best	O(n log n)	O(n log n)
Average	O(n log n)	O(n log n)
Worst	**$O(n^2)$**	**O(n log n)**
Extra array	Usually No	Usually Yes
Average stack	O(log n)	O(log n)
Stable	Usually No	**Yes**
In-place	Usually Yes	Usually No
Pivot required	Yes	No
Merge required	No	Yes
Worst-case guarantee	No	**Yes**

Main difference

Quick Sort:

Partition around pivot

↓

Sort left + right

Merge Sort:

Divide into halves

↓

Sort both halves

↓

Merge them

Advantages of Quick Sort

1. Fast average performance

Average complexity:

O(n log n)

2. In-place partitioning

The standard partitioning approach can rearrange the array without requiring another array of size n.

3. Good cache performance

Quick Sort often works efficiently with arrays because its partitioning accesses elements in a relatively localized manner.

4. Widely useful

It is a fundamental sorting technique and forms the basis of many practical sorting implementations and variations.

5. No merging array required

Unlike Merge Sort, it does not need a separate O(n) merge array in the standard in-place implementation.

25. Disadvantages of Quick Sort

1. Worst-case $O(n^2)$

Poor pivot selection can lead to highly unbalanced partitions.

2. Usually not stable

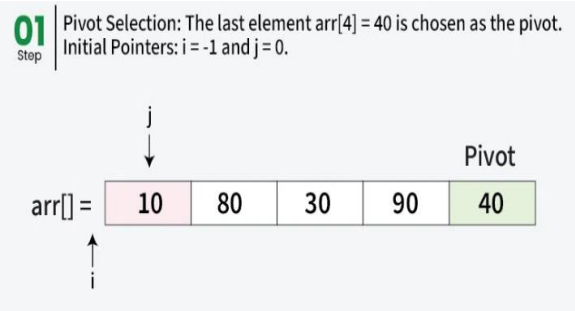
Equal elements may change their relative order.

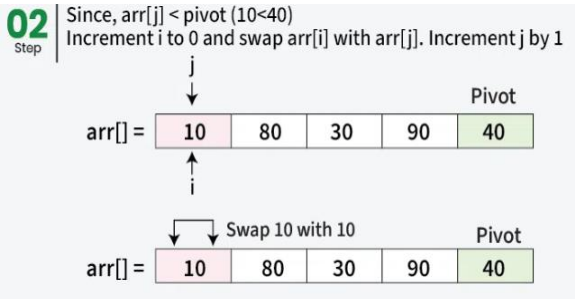
3. Pivot selection matters

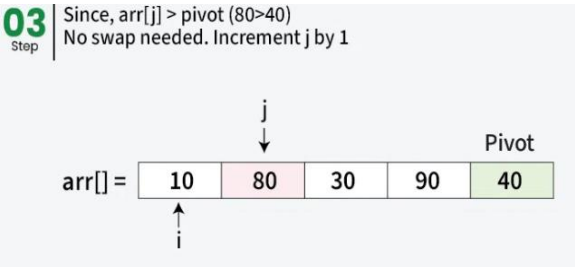
A poor pivot can significantly reduce performance.

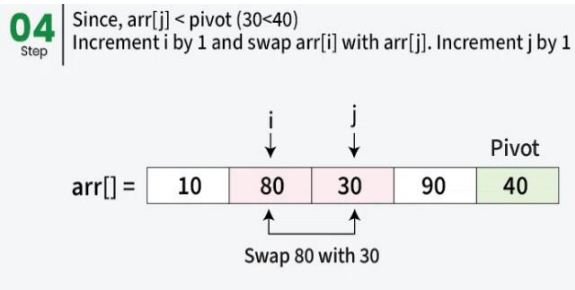
4. Recursive

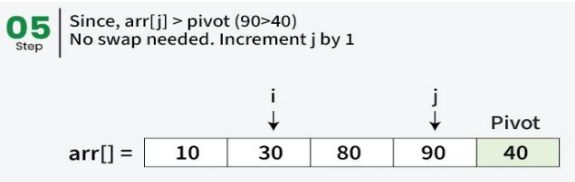
The recursive implementation uses stack memory.

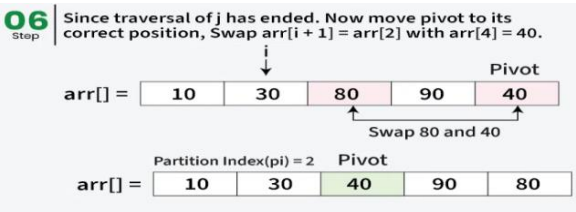












Queues

A **Queue** is a **linear data structure** in which insertion is performed at one end called the **REAR**, and deletion is performed at the other end called the **FRONT**.

Queue follows the principle:

FIFO — First In, First Out

The element inserted first is removed first.

Real-life example

Consider a queue of people waiting at a ticket counter:

Person A → Person B → Person C → Person D

↑ ↑

FRONT REAR

Person A came first, so Person A will be served first.

After A is removed:

Person B → Person C → Person D

↑ ↑

FRONT REAR

2. Basic Terminology

A queue generally uses two pointers/indices:

FRONT

FRONT indicates the position of the **first element** of the queue.

It is used for **deletion**.

REAR

REAR indicates the position where the **next element is inserted** or, depending on the implementation convention, the position of the last element.

It is used for **insertion**.

For example:

FRONT REAR

↓ ↓

```
        +----+----+----+----+
Queue  | 10 | 20 | 30 | 40 |
        +----+----+----+----+
```

Deletion occurs from the **front**:

10 → removed

Insertion occurs at the **rear**:

50 → inserted

3. Basic Operations of a Queue

The main operations are:

Operation	**Meaning**
Enqueue	Insert an element
Dequeue	Delete an element
Peek / Front	View the front element
isEmpty()	Check whether queue is empty
isFull()	Check whether queue is full
Display	Display queue elements

4. Sequential Implementation of Linear Queue

A **linear queue** can be implemented using an array.

Suppose the queue has size 5:

```
Queue = [ _ , _ , _ , _ , _ ]
          0   1   2   3   4
```

Initially:

FRONT = -1

REAR = -1

We use:

MAX = 5

5. Enqueue Operation

Enqueue means inserting an element at the rear of the queue.

Algorithm

```
ENQUEUE(Q, item)
1. If REAR == MAX - 1:
     Print "Queue Overflow"
     Stop
2. If FRONT == -1:
     FRONT = 0
3. REAR = REAR + 1
4. Q[REAR] = item
5. Stop
```

Example of Enqueue

Initially:

```
Queue = [ _ , _ , _ , _ , _ ]
```

FRONT = -1

REAR = -1

Enqueue(10)

```
Queue = [10, _, _, _, _]
          ↑
        FRONT
          ↑
        REAR
```

Now:

FRONT = 0

REAR = 0

Enqueue(20)

```
Queue = [10, 20, _, _, _]
          ↑   ↑
       FRONT REAR
```

Enqueue(30)

```
Queue = [10, 20, 30, _, _]
          ↑        ↑
       FRONT    REAR
```

6. Dequeue Operation

Dequeue means deleting an element from the **front**.

Algorithm

```
DEQUEUE(Q)
1. If FRONT == -1 OR FRONT > REAR:
     Print "Queue Underflow"
     Stop
2. item = Q[FRONT]
3. If FRONT == REAR:
```

```
    FRONT = -1
    REAR = -1
  Else:
    FRONT = FRONT + 1
```

4. Return item

Example

Suppose:

```
Queue = [10, 20, 30, 40, _]
          ↑              ↑
       FRONT          REAR
```

Perform:

DEQUEUE()

10 is removed.

Now:

```
Queue = [10, 20, 30, 40, _]
             ↑           ↑
          FRONT       REAR
```

Logically the queue is:

20 → 30 → 40

FRONT has moved from index 0 to index 1.

7. Peek / Front Operation

The **Peek** operation returns the element at the front without deleting it.

Algorithm

```
PEEK(Q)
1. If FRONT == -1:
    Print "Queue is Empty"
2. Else:
    Return Q[FRONT]
```

For:

```
Queue = [20, 30, 40]
          ↑
       FRONT
```

PEEK() returns:

20

but 20 is not removed.

Python Implementation

```
class LinearQueue:

  def __init__(self, size):
    self.size = size
    self.queue = [None] * size
    self.front = -1
    self.rear = -1

  def enqueue(self, item):

    # Check for overflow
    if self.rear == self.size - 1:
      print("Queue Overflow")
      return

    # First element
    if self.front == -1:
      self.front = 0

    self.rear += 1
    self.queue[self.rear] = item

    print(item, "inserted")

  def dequeue(self):

    # Check for underflow
    if self.front == -1 or self.front >
self.rear:
      print("Queue Underflow")
      return None

    item = self.queue[self.front]

    self.front += 1

    # Queue becomes empty
    if self.front > self.rear:
```

```
            self.front = -1
            self.rear = -1

        return item

    def peek(self):

        if self.front == -1:
            print("Queue is empty")
            return None

        return self.queue[self.front]

    def display(self):

        if self.front == -1:
            print("Queue is empty")
            return

        print("Queue:", end=" ")

        for i in range(self.front, self.rear + 1):
            print(self.queue[i], end=" ")

        print()


# Create queue of size 5
q = LinearQueue(5)

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.enqueue(40)

q.display()

print("Deleted:", q.dequeue())

print("Front element:", q.peek())

q.display()
```

Output

```
10 inserted
20 inserted
30 inserted
40 inserted
Queue: 10 20 30 40
Deleted: 10
Front element: 20
Queue: 20 30 40
```

8. Linear Queue Problem — False Overflow

There is an important limitation of a linear queue implemented using an array.

Suppose the queue has size 5:

```
[10, 20, 30, 40, 50]
 ↑                  ↑
FRONT             REAR
```

Now perform two dequeue operations:

DEQUEUE() → 10

DEQUEUE() → 20

Logically:

30 → 40 → 50

But the array looks like:

```
[10, 20, 30, 40, 50]
      ↑       ↑
    FRONT    REAR
```

There are empty spaces at the beginning:

```
[10, 20, 30, 40, 50]
 X  X      ↑  ↑  ↑
       elements
```

However:

REAR = MAX - 1

So another insertion is not allowed.

This is called **false overflow** or **wasted space**.

Solution

Use a **Circular Queue**.

9. Circular Queue

A **Circular Queue** is a queue in which the last position is logically connected to the first position.

Instead of treating the array as a straight line:

0 → 1 → 2 → 3 → 4

we treat it as circular:

```
    0
  ↗   ↘
 4     1
 ↑     ↓
   3 ← 2
```

After the last index, we return to index 0.

The next position is calculated using:

(REAR + 1) % MAX

10. Circular Queue Example

Suppose:

MAX = 5

Initially:

```
[ _ , _ , _ , _ , _ ]
 0   1   2   3   4
```

Insert:

10, 20, 30, 40, 50

Queue:

```
[10, 20, 30, 40, 50]
 ↑              ↑
FRONT          REAR
```

Now delete two:

10

20

Logically:

30 → 40 → 50

Now insert:

60

Since the queue is circular, REAR moves from index 4 to index 0:

```
[60, _, 30, 40, 50]
 ↑      ↑
REAR   FRONT
```

Then insert 70:

```
[60, 70, 30, 40, 50]
   ↑   ↑
  REAR FRONT
```

The previously unused spaces are reused.

11. Circular Queue Conditions

For a circular queue, the conditions are slightly different.

Queue Empty

A common convention is:

FRONT == -1

Queue Full

(REAR + 1) % MAX == FRONT

This means the next position of REAR is FRONT, so there is no free position.

12. Circular Queue — Enqueue Algorithm

CIRCULAR_ENQUEUE(Q, item)

```
1. If (REAR + 1) % MAX == FRONT:
     Print "Queue Overflow"
     Stop
2. If FRONT == -1:
     FRONT = 0
     REAR = 0
3. Else:
     REAR = (REAR + 1) % MAX
4. Q[REAR] = item
5. Stop
```

13. Circular Queue — Dequeue Algorithm

CIRCULAR_DEQUEUE(Q)

1. If FRONT == -1:
   Print "Queue Underflow"
   Stop
2. item = Q[FRONT]
3. If FRONT == REAR:
   FRONT = -1
   REAR = -1
4. Else:
   FRONT = (FRONT + 1) % MAX
5. Return item

14. Python Implementation of Circular Queue

```
class CircularQueue:
  def __init__(self, size):
    self.size = size
    self.queue = [None] * size
    self.front = -1
    self.rear = -1

  def enqueue(self, item):

    # Check full condition
    if (self.rear + 1) % self.size ==
self.front:
      print("Queue Overflow")
      return

    # First element
    if self.front == -1:
      self.front = 0
      self.rear = 0
    else:
      self.rear = (self.rear + 1) % self.size

    self.queue[self.rear] = item

  def dequeue(self):

    # Check empty condition
    if self.front == -1:
      print("Queue Underflow")
      return None

    item = self.queue[self.front]

    # Only one element
    if self.front == self.rear:
      self.front = -1
      self.rear = -1
    else:
      self.front = (self.front + 1) % self.size

    return item

def peek(self):

  if self.front == -1:
    return None

  return self.queue[self.front]

def display(self):

  if self.front == -1:
    print("Queue is empty")
    return

  i = self.front

  while True:
    print(self.queue[i], end=" ")

    if i == self.rear:
      break

    i = (i + 1) % self.size
```

```
        print()


# Example
q = CircularQueue(5)

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.enqueue(40)

q.display()

print("Deleted:", q.dequeue())
print("Deleted:", q.dequeue())

q.enqueue(50)
q.enqueue(60)

q.display()
```

Possible output:

10 20 30 40

Deleted: 10

Deleted: 20

30 40 50 60

15. Priority Queue

A **Priority Queue** is a special type of queue in which each element is associated with a **priority**.

Unlike a normal FIFO queue, the element with the **higher priority is served first**.

For elements having the same priority, FIFO order can be maintained.

Example

Suppose a hospital emergency department has:

Patient	Priority
A	2
B	5
C	1
D	4

If a larger number means higher priority:

B (5)

D (4)

A (2)

C (1)

The patient with priority 5 is served first.

---

16. Types of Priority Queue

Two common types are:

1. Max-Priority Queue

Element with the **highest priority** is removed first.

Example:

Priority:

10 > 8 > 5 > 2

10 is served first.

2. Min-Priority Queue

Element with the **lowest priority** is removed first.

Example:

Priority:

2 < 5 < 8 < 10

2 is served first.

Priority Queue — Enqueue and Dequeue Algorithms

A **Priority Queue** is a queue in which each element has an associated priority.

Unlike a normal queue, deletion is based on **priority**.

For example, assuming **larger number = higher priority**:

Element	Priority
A	2
B	5

C 1
D 4
The deletion order will be:
B → D → A → C
5 4 2 1

---

5. Priority Queue Enqueue
In a simple **unsorted-array implementation**, enqueue is easy.
We simply insert the new element at the end of the array/list along with its priority.
Algorithm
ENQUEUE(PQ, item, priority)

1. If queue is full:
   Print "Priority Queue Overflow"
   Return

2. Create an element:
   NEW = (item, priority)

3. Insert NEW at the REAR of the queue

4. REAR = REAR + 1

5. Stop
Example
Initially:
Priority Queue:
[(A, 2), (B, 5), (C, 1)]
Perform:
ENQUEUE(D, 4)
Now:
[(A, 2), (B, 5), (C, 1), (D, 4)]
Notice that we **do not need to arrange the elements during enqueue**.

---

6. Priority Queue Dequeue
During deletion, we search for the element having the **highest priority**.
Assume larger priority number means higher priority.
Algorithm
DEQUEUE(PQ)

1. If PQ is empty:
   Print "Priority Queue Underflow"
   Return

2. highest = 0

3. For i = 1 to REAR:

   If PQ[i].priority > PQ[highest].priority:

      highest = i

4. ITEM = PQ[highest]

5. Remove PQ[highest] from the queue

6. Return ITEM

---

7. Priority Queue Example
Suppose:
PQ = [(A,2), (B,5), (C,1), (D,4)]
Dequeue 1
Compare priorities:
A → 2
B → 5 ← highest
C → 1
D → 4
Therefore:
DEQUEUE() → B
Remaining:
[(A,2), (C,1), (D,4)]

Dequeue 2

Compare:

A → 2

C → 1

D → 4 ← highest

Therefore:

DEQUEUE() → D

Remaining:

[(A,2), (C,1)]

Dequeue 3

A → 2 ← highest

C → 1

Therefore:

DEQUEUE() → A

Finally:

DEQUEUE() → C

So the deletion order is:

B → D → A → C

17. Priority Queue Implementation

A priority queue can be implemented using:

1. Array
2. Linked List
3. Heap

A **heap** is commonly used when efficient priority-queue operations are required.

18. Simple Priority Queue Using List

Consider:

[(A, 2), (B, 5), (C, 1), (D, 4)]

where the second value represents priority.

We can implement a simple priority queue by finding the highest-priority element during deletion.

```
class PriorityQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item, priority):
        self.queue.append((item, priority))

    def dequeue(self):

        if not self.queue:
            print("Priority Queue is empty")
            return None

        # Find highest priority
        highest = 0

        for i in range(1, len(self.queue)):
            if self.queue[i][1] >
self.queue[highest][1]:
                highest = i

        return self.queue.pop(highest)

    def display(self):
        print(self.queue)


pq = PriorityQueue()

pq.enqueue("A", 2)
pq.enqueue("B", 5)
pq.enqueue("C", 1)
pq.enqueue("D", 4)

print("Deleted:", pq.dequeue())
print("Deleted:", pq.dequeue())
```

Output:

Deleted: ('B', 5)

Deleted: ('D', 4)

19. Priority Queue Using Heap

Python provides the heapq module for implementing a **min-priority queue** efficiently.

```
import heapq

pq = []

heapq.heappush(pq, (2, "A"))
heapq.heappush(pq, (5, "B"))
heapq.heappush(pq, (1, "C"))
heapq.heappush(pq, (4, "D"))

print(heapq.heappop(pq))
print(heapq.heappop(pq))
```

Output:

```
(1, 'C')
(2, 'A')
```

Here, the **smallest priority value is removed first**.

For a max-priority queue, negative priorities can be used:

```
heapq.heappush(pq, (-5, "B"))
```

---

20. Queue Implementations and Complexity

Implementation	Enqueue	Dequeue	Peek
Linear Array	O(1)	O(1)	O(1)
Circular Array	O(1)	O(1)	O(1)
Unsorted Priority Queue	O(1)	O(n)	O(n)
Sorted Priority Queue	O(n)	O(1)	O(1)
Binary Heap Priority Queue	O(log n)	O(log n)	O(1)

The exact complexity can vary with the implementation details.

---

21. Applications of Queues

Queues are used whenever items need to be processed in an ordered manner.

1. CPU Scheduling

Processes waiting for CPU time can be maintained in queues.

Example:

P1 → P2 → P3 → P4

The operating system selects processes according to the scheduling policy.

---

2. Printer Queue

When multiple users send documents to a printer:

```
Document A
   ↓
Document B
   ↓
Document C
   ↓
Printer
```

Documents can be processed in queue order.

---

3. Network Data Packets

Routers and network devices use queues to temporarily hold packets waiting for transmission.

```
Packet 1
Packet 2
Packet 3
  ↓
Network Interface
```

---

4. Breadth-First Search (BFS)

A queue is a fundamental data structure used in **Breadth-First Search** of graphs.
Example:
Start
↓
Visit neighboring vertices
↓
Put them into queue
↓
Process them one by one

---

5. Customer Service Systems
Customers can be placed in a queue:
Customer 1 → Customer 2 → Customer 3

---

6. Operating Systems
Queues are used in:

- Process scheduling
- I/O requests
- Device management
- Job scheduling

---

7. Simulation
Queues are used to simulate:

- Traffic systems
- Bank counters
- Airport check-in
- Call centers
- Hospital service systems

---

8. Interrupt Handling
Operating systems can maintain queues of tasks or requests that need processing.

---

9. Messaging Systems
Messages waiting to be processed can be maintained in queues.
For example:
Message 1 → Message 2 → Message 3
↓
Message Processor

---

22. Linear Queue vs Circular Queue vs Priority Queue

Feature	**Linear Queue**	**Circular Queue**	**Priority Queue**
Principle	FIFO	FIFO	Priority-based
Insertion	Rear	Rear	Based on implementation
Deletion	Front	Front	Highest/lowest priority
Reuses empty spaces	No	**Yes**	Depends
Main issue	False overflow	More complex implementation	Priority management
Common use	Simple waiting line	Buffers, scheduling	CPU scheduling, emergency systems

---

23. Important Queue Conditions
Queue Overflow
Occurs when we try to insert into a full queue.
ENQUEUE → FULL QUEUE
↓
OVERFLOW
Queue Underflow

Occurs when we try to delete from an empty queue.

DEQUEUE → EMPTY QUEUE

↓

UNDERFLOW

---

24. Queue vs Stack

This is an important exam comparison.

Feature	**Stack**	**Queue**
Principle	**LIFO**	**FIFO**
Insertion	Top	Rear
Deletion	Top	Front
Main operations	Push, Pop	Enqueue, Dequeue
Example	Stack of plates	Line of people
Applications	Function calls, undo	Scheduling, BFS